01NAEX - Lecture 06
The 2^k Factorial Design II: a Single Replicate

Jiri Franc

Czech Technical University
Faculty of Nuclear Sciences and Physical Engineering
Department of Mathematics

The 2^k Factorial Design

Last lecture: the 2^k factorial design with n replicates

- effects from the table of plus and minus signs, effect $= 2\hat{\beta}$,
- ANOVA, model refinement, lack of fit, hierarchy, confidence intervals of effects,
- coded and natural variables, response surface and contour plots.

Today: a single replicate of the 2^k design (Montgomery, Sections 6.5–6.6)

- normal (Daniel) and half-normal probability plots of the effects,
- Lenth's method and the Pareto plot of the effects,
- projection of the design, the final model and its interpretation,
- an outlier, a data transformation and dispersion effects.

General 2^k Factorials (Overview & Notation)

Design: k two-level factors, levels coded $-1, +1$, $N = n2^k$ runs.

Regression model in coded variables:

$$y = \beta_0 + \sum_i \beta_i x_i + \sum_{i<j} \beta_{ij} x_i x_j + \cdots + \beta_{12\ldots k} x_1 \cdots x_k + \varepsilon.$$

The general 2^k factorial design has

- k main effects,
- $\binom{k}{2}$ two-factor interactions, $\binom{k}{3}$ three-factor interactions, $\ldots$,
- 1 k-factor interaction: $2^k - 1$ effects in total.

Effect estimates (orthogonal coding, column of signs $s_r \in \{-1, +1\}$):

- Contrast $C_E = \sum_{\text{runs}} s_r \, y_r$ (sum over the treatment totals),
- Effect $\hat{E} = C_E/(n2^{k-1})$, $\hat{\beta} = \hat{E}/2$,
- Sum of squares $SS_E = C_E^2/(n2^k)$, one degree of freedom.

A Single Replicate of the 2^k Design

- With many factors even one replicate is expensive: $2^4 = 16$, $2^5 = 32$, $2^6 = 64$ runs. A single replicate (unreplicated factorial) is common in screening experiments.
- If the factor levels are spaced too closely, the noise may overwhelm the signal. More aggressive spacing of the levels is usually best.
- **Problem:** the full model has 2^k parameters and we have 2^k observations, so there are **zero degrees of freedom for error**. There is no internal estimate of σ^2 and no F-tests.
- **Sparsity of effects principle:** most systems are dominated by some of the main effects and low-order interactions; most high-order interactions are negligible.
- What we can do:
  - pool the (presumably negligible) high-order interactions into an error term – risky if one of them is active;
  - look at the **normal (Daniel) or half-normal probability plot** of the effects;
  - use **Lenth's method** (a robust pseudo standard error of the effects);
  - **project** the design onto the active factors – then we get replicates.

Pilot Plant Filtration Rate Experiment (Example 6.2)

A 2^4 factorial design was used to investigate the effects of four factors on the filtration rate of a resin for a chemical process plant: **A** temperature, **B** pressure, **C** concentration of formaldehyde, **D** stirring rate. The $2^4 = 16$ runs were made in random order; the aim is to maximize the filtration rate.

Run Number	Factor A	B	C	D	Run Label	Filtration Rate (gal/h)
1	−	−	−	−	(1)	45
2	+	−	−	−	*a*	71
3	−	+	−	−	*b*	48
4	+	+	−	−	*ab*	65
5	−	−	+	−	*c*	68
6	+	−	+	−	*ac*	60
7	−	+	+	−	*bc*	80
8	+	+	+	−	*abc*	65
9	−	−	−	+	*d*	43
10	+	−	−	+	*ad*	100
11	−	+	−	+	*bd*	45
12	+	+	−	+	*abd*	104
13	−	−	+	+	*cd*	75
14	+	−	+	+	*acd*	86
15	−	+	+	+	*bcd*	70
16	+	+	+	+	*abcd*	96

Filtration Rate – Effects from the Table of Signs in Python

```
import itertools
import numpy as np
import pandas as pd

url = ("https://raw.githubusercontent.com/francji1/01NAEX/"
       "main/data/"
       "Pilot_Plant_Filtration_Rate_Experiment.csv")
rate = pd.read_csv(url, sep=";")    # A, B, C, D, Rate

X = rate[list("ABCD")]
eff = {}
for r in range(1, 5):
    for t in itertools.combinations("ABCD", r):
        s = X[list(t)].prod(axis=1)     # column of signs
        # effect = contrast / 2^(k-1)
        eff["".join(t)] = (s * rate["Rate"]).sum() / 8
eff = pd.Series(eff)
eff.sort_values()

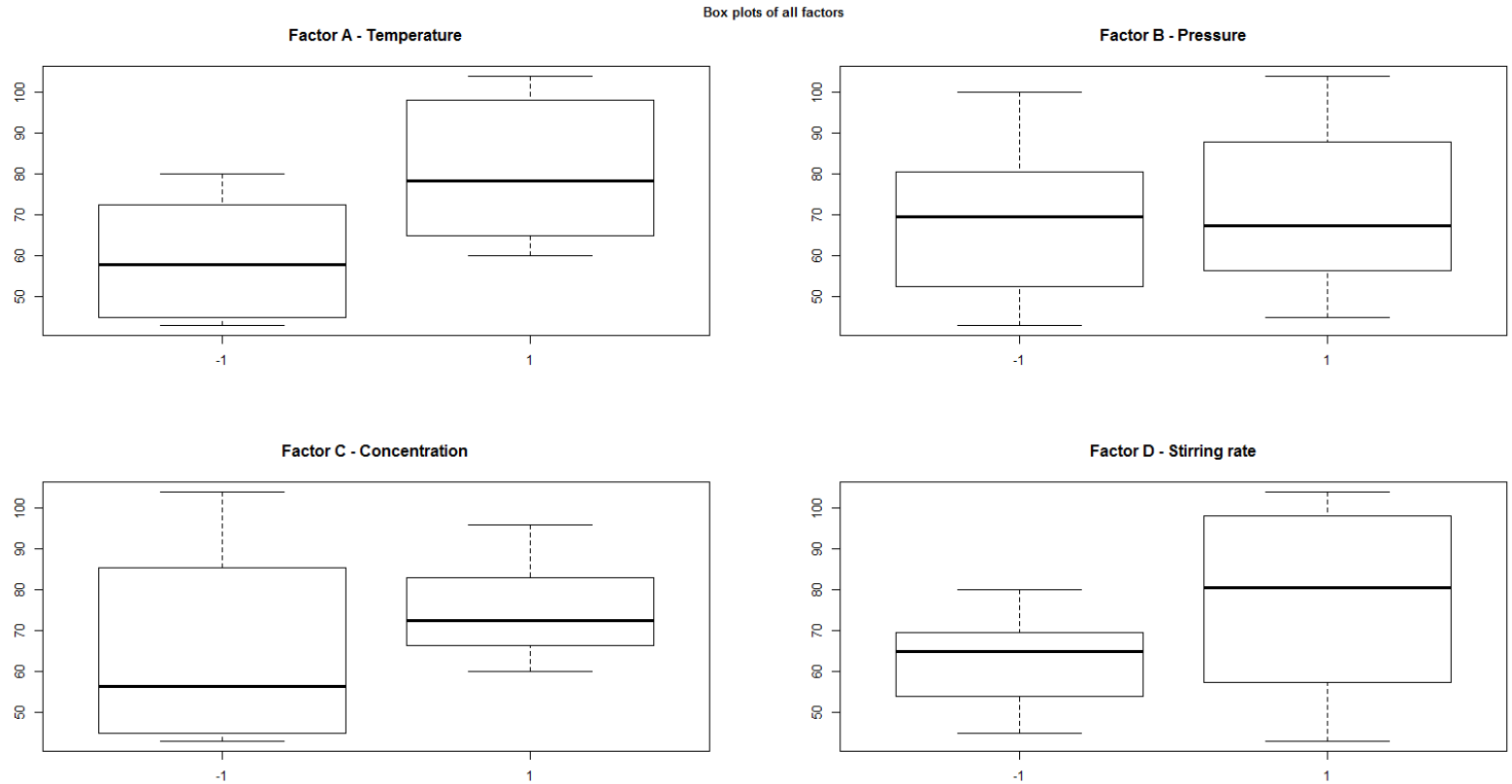
AC -18.125  BCD -2.625  ACD -1.625  CD -1.125  BD -0.375
AB   0.125  ABCD 1.375  ABC  1.875  BC  2.375  B   3.125
ABD  4.125  C    9.875  D   14.625  AD 16.625  A  21.625
```

The effects *A*, *C*, *D*, *AC* and *AD* are much larger than the others.

Filtration Rate – Box Plots

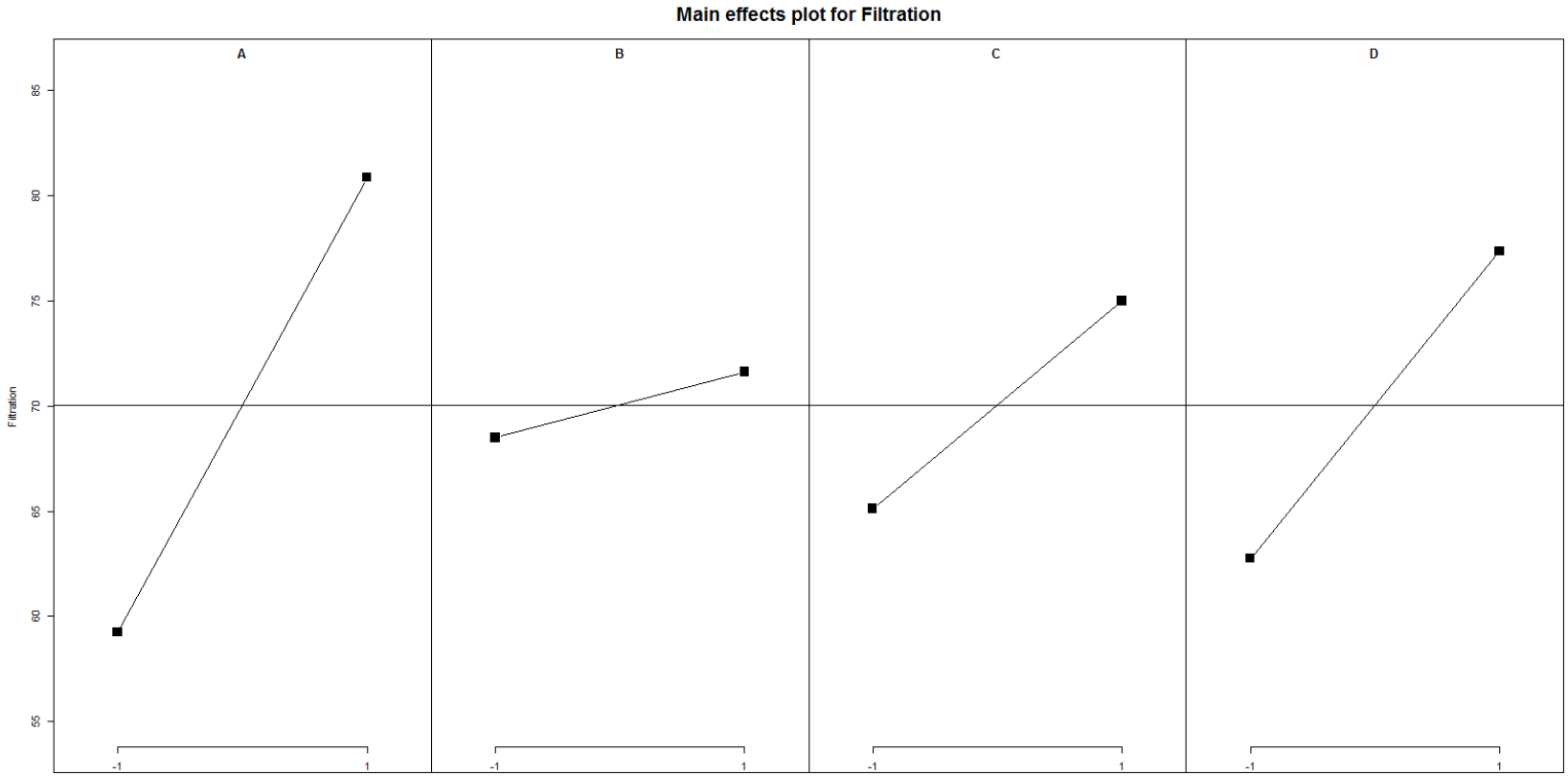
Box plots of the response for the two levels of each factor:

```
for f in "ABCD":
    rate.boxplot(column="Rate", by=f)
```



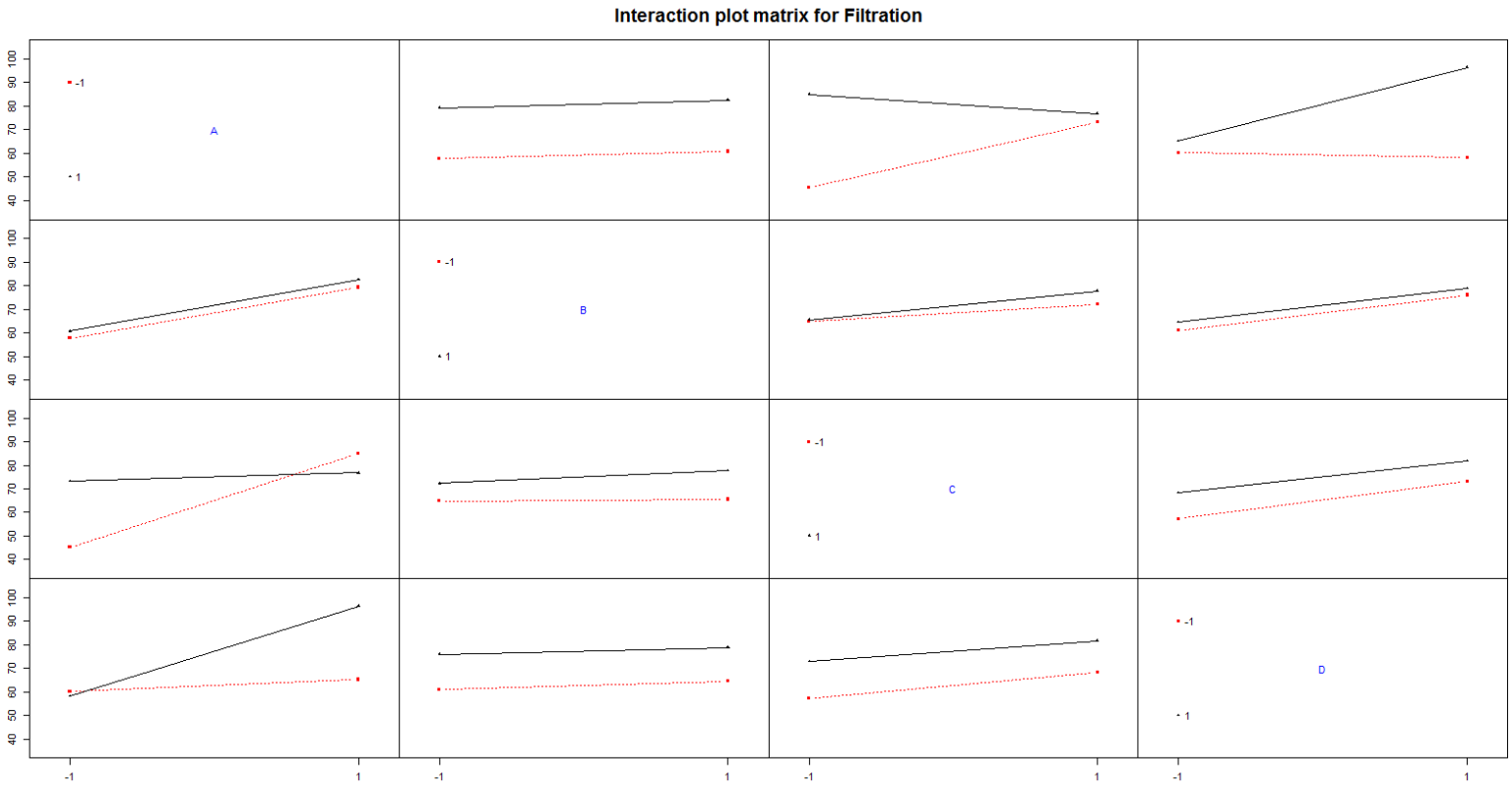
Filtration Rate – Main Effects Plot

```
fig, ax = plt.subplots(1, 4, sharey=True)
for a, f in zip(ax, "ABCD"):
    rate.groupby(f)["Rate"].mean().plot(ax=a, marker="o")
```



Filtration Rate – Interaction Plots

```
from statsmodels.graphics.factorplots import (
    interaction_plot)
for f1, f2 in itertools.combinations("ABCD", 2):
    interaction_plot(rate[f1], rate[f2], rate["Rate"])
```



Filtration Rate – the Full Model Has No Error Term

```
import statsmodels.formula.api as smf
from statsmodels.stats.anova import anova_lm

full = smf.ols("Rate ~ A*B*C*D", data=rate).fit()
full.df_resid            # 0.0 -> no estimate of sigma^2
2 * full.params.drop("Intercept")    # effects = 2 * coef
anova_lm(full).round(2).head(6)
```

```
      df    sum_sq  mean_sq    F  PR(>F)
A    1.0  1870.56  1870.56  0.0     NaN
B    1.0    39.06    39.06  0.0     NaN
A:B  1.0     0.06     0.06  0.0     NaN
C    1.0   390.06   390.06  0.0     NaN
A:C  1.0  1314.06  1314.06  0.0     NaN
B:C  1.0    22.56    22.56  0.0     NaN
```

- The sums of squares are correct ($SS = 16\,\hat{E}^2/4$), but there are 16 parameters, 16 observations and **0 residual degrees of freedom**.
- Careful: statsmodels prints $F = 0.0$ (not missing) here. These are not test results.

Analysis of Significant Effects

We cannot use the usual F-tests because

- ANOVA gives the sums of squares but no error estimate,
- the regression gives the effect estimates but no standard errors,
- we fit a model with 2^k parameters to 2^k observations, leaving 0 degrees of freedom for the error.

What we know about the effect estimates $\hat{E}_1, \ldots, \hat{E}_{2^k-1}$:

- they are linear combinations of independent normal observations with orthogonal coefficients, hence **independent and normal** with common variance $V(\hat{E}) = 4\sigma^2/N$ ($N = 2^k$),
- if an effect is **negligible** (true value 0), its estimate is $N(0, 4\sigma^2/N)$.

So the negligible effects behave like a random sample from a normal distribution with mean 0, while the **active effects are outliers**. Let us plot the effects on a normal probability plot and look for the outliers.

Normal (Daniel) and Half-Normal Plots of the Effects

Normal probability plot of the effects (Daniel, 1959): sort the effects $\hat{E}_{(1)} \leq \cdots \leq \hat{E}_{(m)}$, $m = 2^k - 1$, and plot them against the normal quantiles $\Phi^{-1}\big((j - 0.5)/m\big)$.

1. **Negligible effects** lie along a straight line through (approximately) zero.
2. **Important effects** lie far from the line.

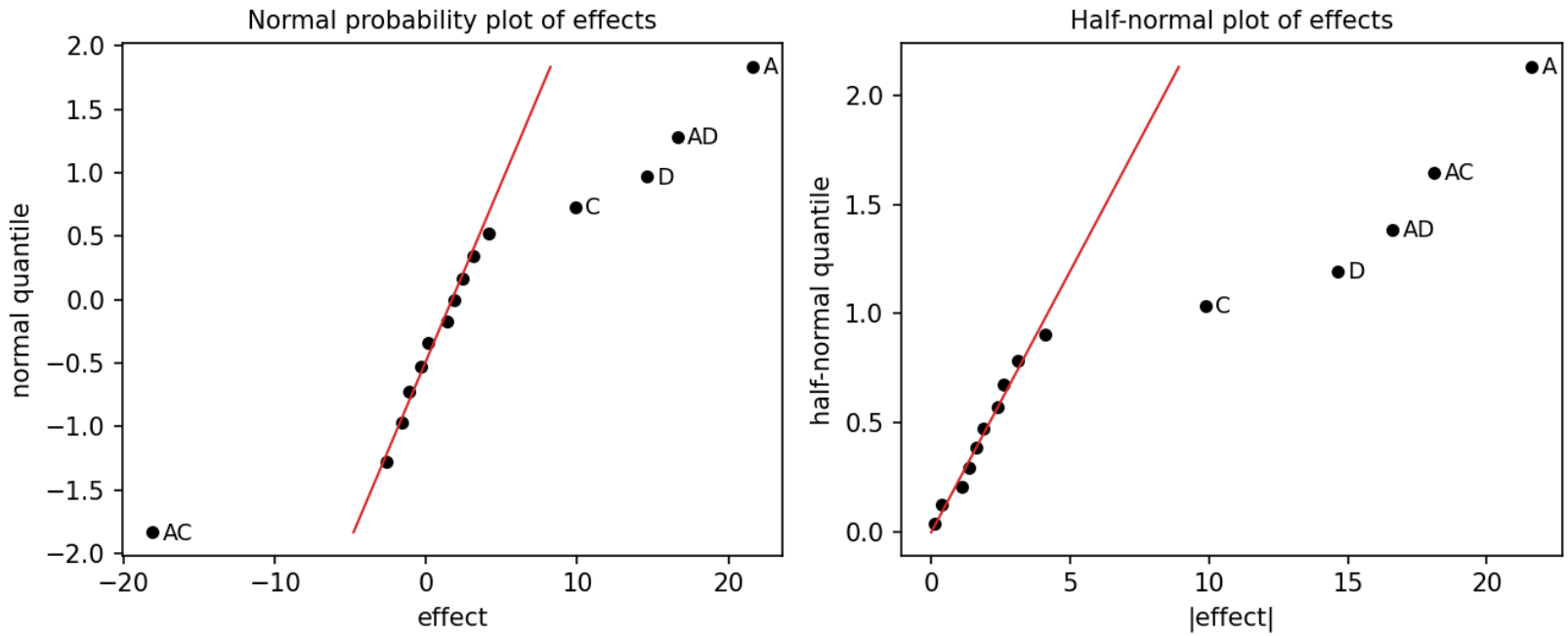
Half-normal plot: plot the absolute values $|\hat{E}|_{(j)}$ against the half-normal quantiles $\Phi^{-1}\big(0.5 + 0.5(j - 0.5)/m\big)$.

- The straight line always passes through the origin and should pass close to the fiftieth percentile of the data.
- There is no sign ambiguity (a large negative effect is not hidden in the lower tail). Better for interpretation with few effects (for example an 8-run design).

Draw the line through the **small** effects (by eye, or fitted to the smaller half), not through all of them: the active effects would pull it away.

Filtration Rate – Normal and Half-Normal Plots in Python

```
from scipy import stats
e = eff.sort_values()
p = (np.arange(1, 16) - 0.5) / 15
plt.scatter(e, stats.norm.ppf(p))          # normal plot
a = eff.abs().sort_values()
plt.scatter(a, stats.halfnorm.ppf(p))      # half-normal
```



Tentative model: *A*, *C*, *D*, *AC*, *AD*. Factor *B* (pressure) does not appear at all.

Lenth's Method

Lenth (1989): an analytical method with good power to detect active effects. Let $c_1, \ldots, c_m$ be the contrasts (effect estimates) of interest; for an unreplicated 2^k design $m = 2^k - 1$. The variance of a contrast is estimated from the smallest (in absolute value) contrast estimates:

$$s_0 = 1.5 \cdot \text{median}\left(|c_j|\right), \qquad PSE = 1.5 \cdot \text{median}\left(|c_j| \ : \ |c_j| < 2.5 s_0\right).$$

PSE is the **pseudo standard error**; it is a reasonable estimator of the standard deviation of a contrast when only a few effects are active (the factor 1.5 makes the median of $|N(0, \tau^2)|$ consistent for τ).

An individual contrast is compared to the **margin of error**

$$ME = t_{0.025,d}\, PSE, \qquad d = \frac{m}{3},$$

a group of contrasts to the **simultaneous margin of error**

$$SME = t_{\gamma,d}\, PSE, \qquad \gamma = \frac{1 + 0.95^{1/m}}{2}$$

(the γ-quantile of the t distribution, Bonferroni-like correction for m contrasts).

Filtration Rate – Lenth's Method in Python

```
def lenth(effects, alpha=0.05):
    c = np.abs(np.asarray(effects))
    m = len(c)
    s0 = 1.5 * np.median(c)
    pse = 1.5 * np.median(c[c < 2.5 * s0])
    d = m / 3
    me = stats.t.ppf(1 - alpha / 2, d) * pse
    gamma = (1 + (1 - alpha) ** (1 / m)) / 2
    sme = stats.t.ppf(gamma, d) * pse
    return pse, me, sme

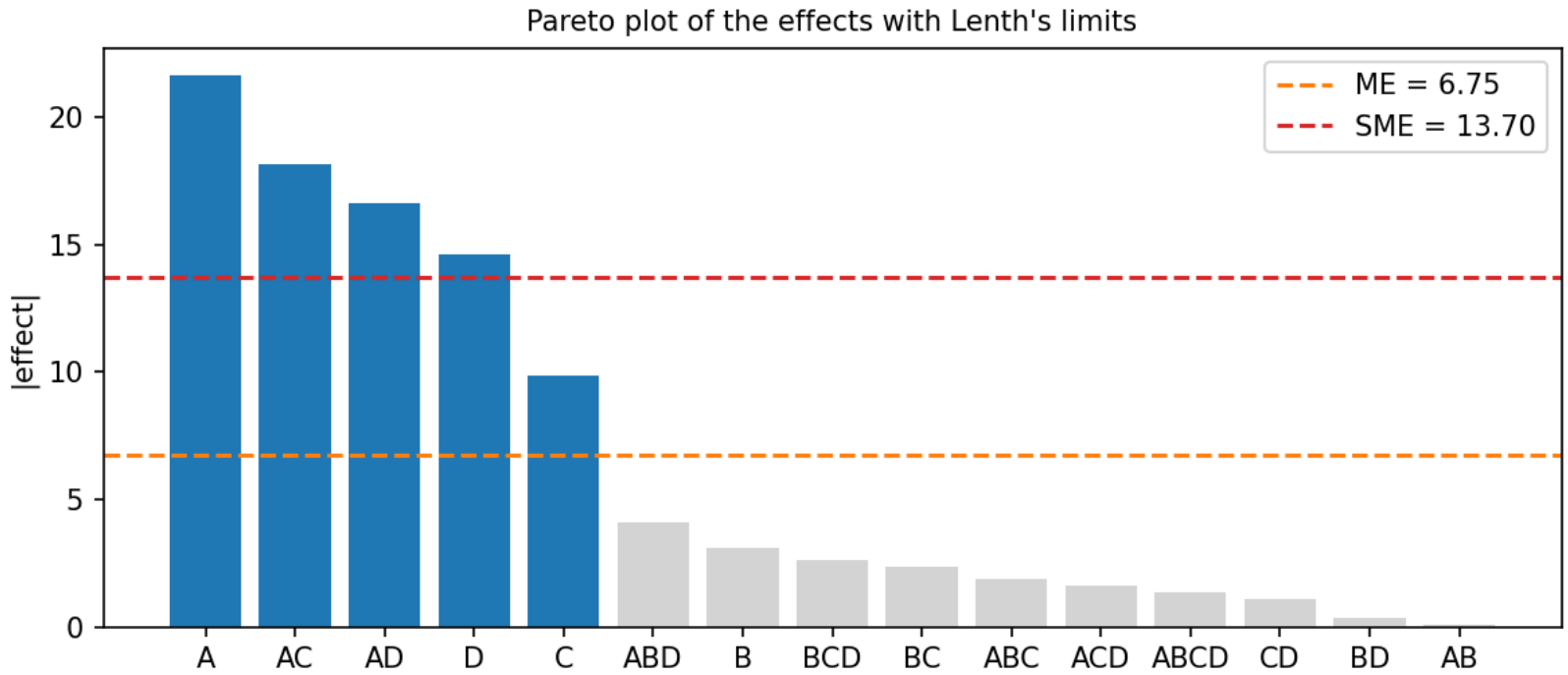
pse, me, sme = lenth(eff)
print(f"PSE = {pse:.3f}, ME = {me:.3f}, SME = {sme:.3f}")
```

```
PSE = 2.625, ME = 6.748, SME = 13.699
```

- $s_0 = 1.5 \times 2.625 = 3.94$, effects below $2.5 s_0 = 9.84$ have median 1.75, so $PSE = 1.5 \times 1.75 = 2.625$; $t_{0.025,5} = 2.571$, $t_{\gamma,5} = 5.219$.
- $|\hat{E}| > SME$: A (21.6), AC (-18.1), AD (16.6), D (14.6). C (9.9) exceeds only ME, but because AC is clearly active, we keep C in the model (hierarchy).

Filtration Rate – Pareto Plot of the Effects

Bars of $|\hat{E}_j|$ sorted by size with the *ME* and *SME* lines (the plot used by JMP and by R packages `qualityTools`, `FrF2`).



Lenth's method gives the same answer as the normal probability plot.

Lenth's Method – Remarks

- Several authors observed that the original *ME* and *SME* are **too conservative** (little power). Larntz and Whitcomb (1998) suggest multipliers calibrated by simulation (Montgomery, Section 6.5):

Number of contrasts m	7	15	31
Original ME	3.764	2.571	2.218
Adjusted ME	2.295	2.140	2.082
Original SME	9.008	5.219	4.218
Adjusted SME	4.891	4.163	4.030

- JMP computes p-values of the Lenth statistics $\hat{E}_j/PSE$ by simulation under the null model (no active effect, 10 000 samples).
- Use Lenth's method as a **supplement** to the normal probability plot, not as its replacement.
- Both methods rely on effect sparsity. If half of the effects are active, *PSE* is inflated and the plot has no straight part.

Daniel vs Half-Normal vs Lenth (Summary When No Replication)

Daniel (normal QQ): plot the effect estimates $\hat{E}$ vs normal quantiles. Active effects depart from the line; the sign of the effect is visible.

Half-normal: plot $|\hat{E}|$ vs half-normal quantiles; no sign ambiguity, the line goes through the origin.

Lenth PSE & Pareto: robust pseudo-σ from the small effects,

$$s_0 = 1.5\,\text{median}(|c_j|),\ PSE = 1.5\,\text{median}\{|c_j| : |c_j| < 2.5 s_0\},$$

then thresholds

$$ME = t_{0.025,d}\,PSE,\ SME = t_{\gamma,d}\,PSE,\ d = m/3.$$

Build a Pareto chart of $|c_j|$ with the *ME* and *SME* lines.

Always: respect the hierarchy, check the residuals of the final model, and think about the engineering meaning.

Filtration Rate – Projection into a 2^3 Design

Factor *B* (pressure) and all its interactions are negligible. Dropping *B* **projects** the single replicate of the 2^4 into a 2^3 design in *A*, *C*, *D* with **two replicates**: now we have 8 degrees of freedom for pure error.

```
m3 = smf.ols("Rate ~ A*C*D", data=rate).fit()
anova_lm(m3).round(4)
```

	df	sum_sq	mean_sq	F	PR(>F)
A	1.0	1870.5625	1870.5625	83.3677	0.0000
C	1.0	390.0625	390.0625	17.3844	0.0031
A:C	1.0	1314.0625	1314.0625	58.5655	0.0001
D	1.0	855.5625	855.5625	38.1309	0.0003
A:D	1.0	1105.5625	1105.5625	49.2730	0.0001
C:D	1.0	5.0625	5.0625	0.2256	0.6475
A:C:D	1.0	10.5625	10.5625	0.4708	0.5120
Residual	8.0	179.5000	22.4375	NaN	NaN

In general a single replicate of a 2^k design projects into a full factorial in any $r < k$ of the factors with 2^{k-r} replicates. Projection is useful only if the dropped factors are really negligible.

Filtration Rate – Final Model

```
final = smf.ols("Rate ~ A + C + D + A:C + A:D",
                data=rate).fit()
anova_lm(final).round(4)
```

	df	sum_sq	mean_sq	F	PR(>F)
A	1.0	1870.5625	1870.5625	95.8648	0.0000
C	1.0	390.0625	390.0625	19.9904	0.0012
D	1.0	855.5625	855.5625	43.8469	0.0001
A:C	1.0	1314.0625	1314.0625	67.3447	0.0000
A:D	1.0	1105.5625	1105.5625	56.6592	0.0000
Residual	10.0	195.1250	19.5125	NaN	NaN

The error mean square 19.51 now has 10 degrees of freedom: the pure error (8 df, $SS = 179.5$) plus the dropped CD and ACD (2 df, $SS = 15.6$). The sums of squares do not depend on the other terms in the model (orthogonal design).

Filtration Rate – Regression Model

```
print(final.summary().tables[1])
```

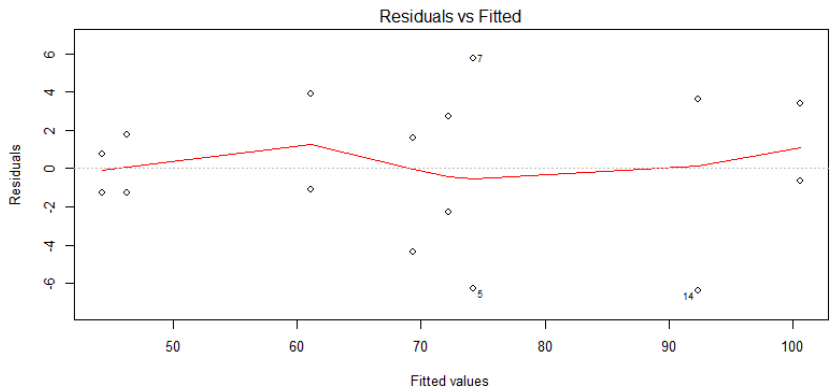
	coef	std err	t	P>|t|
Intercept	70.0625	1.104	63.444	0.000
A	10.8125	1.104	9.791	0.000
C	4.9375	1.104	4.471	0.001
D	7.3125	1.104	6.622	0.000
A:C	-9.0625	1.104	-8.206	0.000
A:D	8.3125	1.104	7.527	0.000

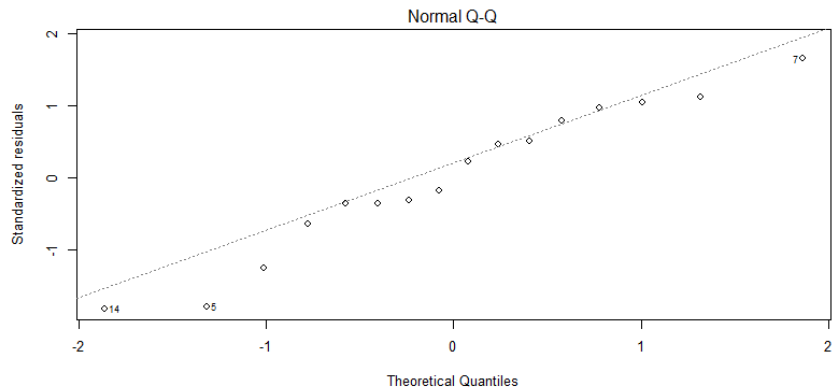
$$\hat{y} = 70.06 + 10.81\, x_1 + 4.94\, x_3 + 7.31\, x_4 - 9.06\, x_1 x_3 + 8.31\, x_1 x_4$$

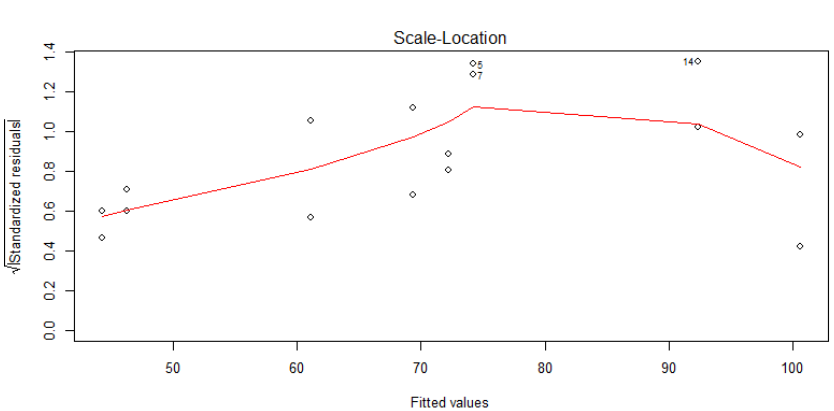
- $\hat{\sigma} = \sqrt{MS_E} = 4.417$, $se(\hat{\beta}) = \sqrt{19.51/16} = 1.104$, $R^2 = 0.966$, $R^2_{adj} = 0.949$.
- The coefficients are half of the effects (21.625/2 = 10.81, . . .).
- The highest predicted rate is at $A = +$, $C = -$, $D = +$: $\hat{y} = 100.6$ gal/h (observed $ad = 100$ and $abd = 104$). High temperature and stirring rate with low concentration of formaldehyde.

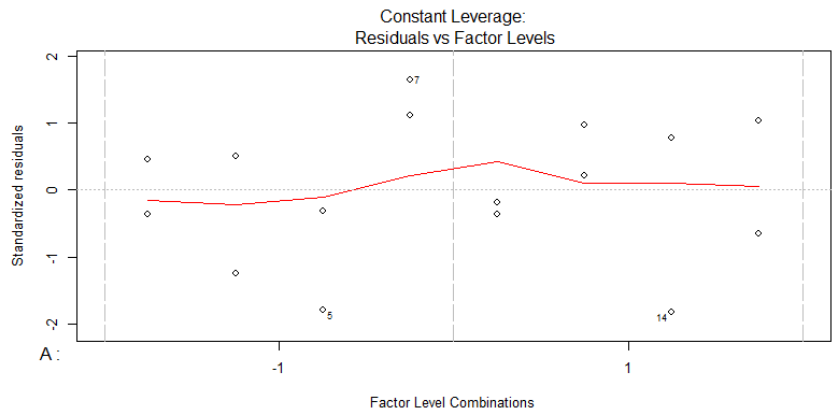
Filtration Rate – Model Validation

```
sm.qqplot(final.resid, line="s")
plt.scatter(final.fittedvalues, final.resid)
stats.shapiro(final.resid)          # W = 0.953, p = 0.55
```





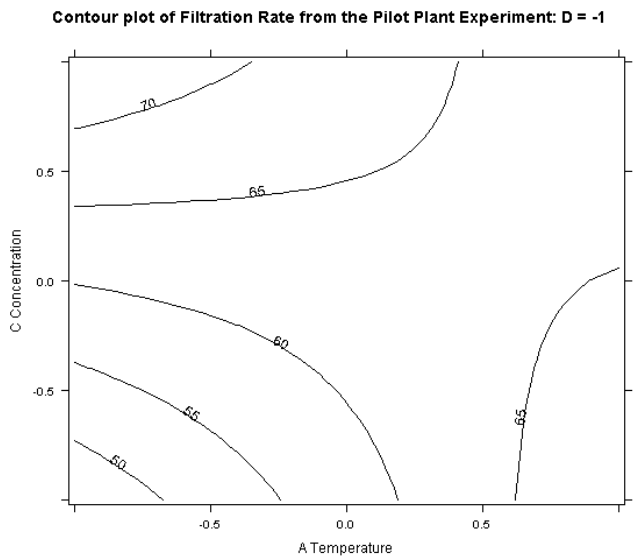


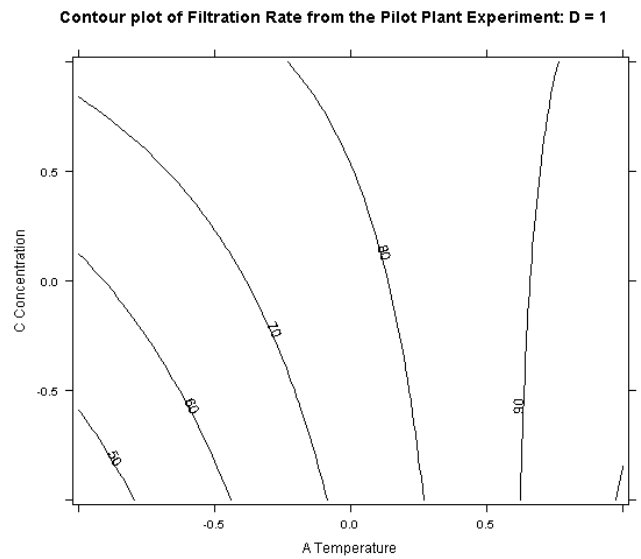


No pattern in the residuals; plot them also against each factor (including the dropped *B*) and against the run order.

Filtration Rate – Contour Plots in Python

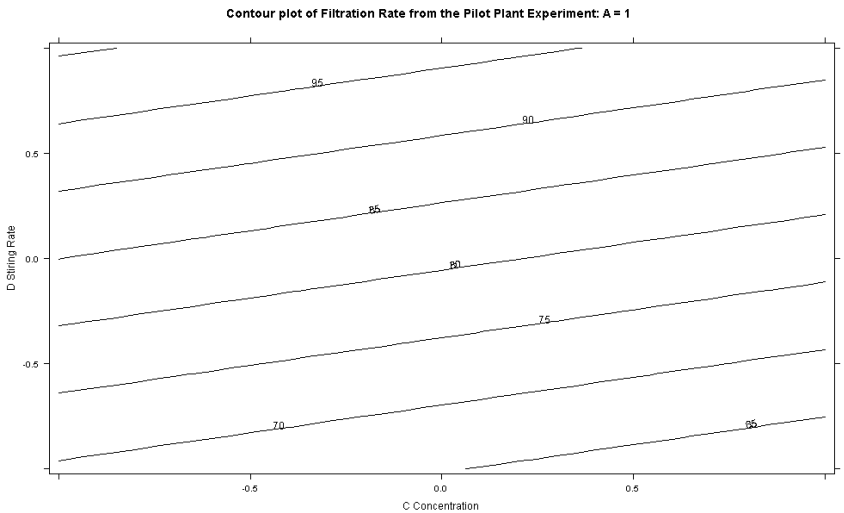
```
g = np.linspace(-1, 1, 41)
A, C = np.meshgrid(g, g)
fig, ax = plt.subplots(1, 2, figsize=(9, 4))
for a, d in zip(ax, [-1, 1]):          # fix D at -1 and +1
    grid = pd.DataFrame({"A": A.ravel(), "C": C.ravel(),
                         "D": d})
    Z = final.predict(grid).to_numpy().reshape(A.shape)
    cs = a.contour(A, C, Z, levels=10)
    a.clabel(cs)
    a.set(xlabel="A temperature", ylabel="C concentration",
          title=f"D = {d}")
```

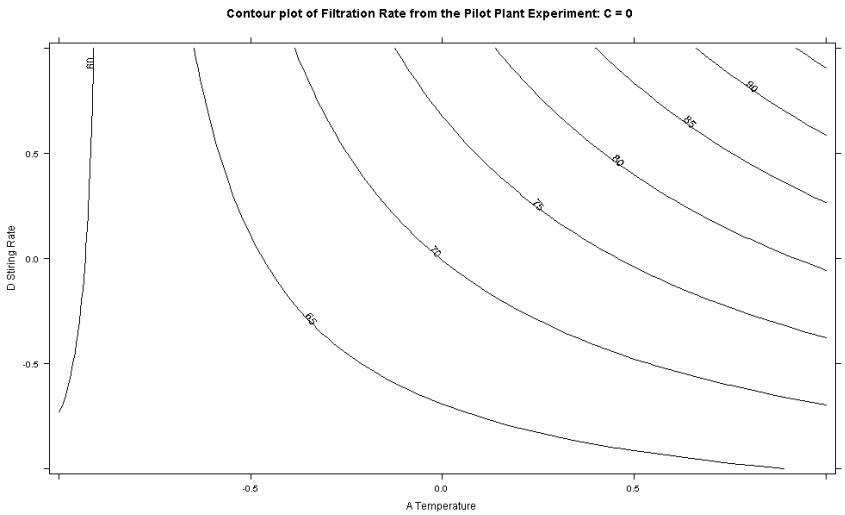




Filtration Rate – Contour Plots

Contour plots in the $C \times D$ plane ($A = +1$) and in the $A \times D$ plane ($C = 0$):





- The highest filtration rate is in the corner $A = +$, $C = -$, $D = +$ (high temperature, low concentration, high stirring rate).
- The model is linear with interactions; outside the cube (extrapolation) it is not supported by the data.

Filtration Rate – Pooling the Highest-Order Interaction?

Another approach: omit only the *ABCD* interaction and use it as the error term.

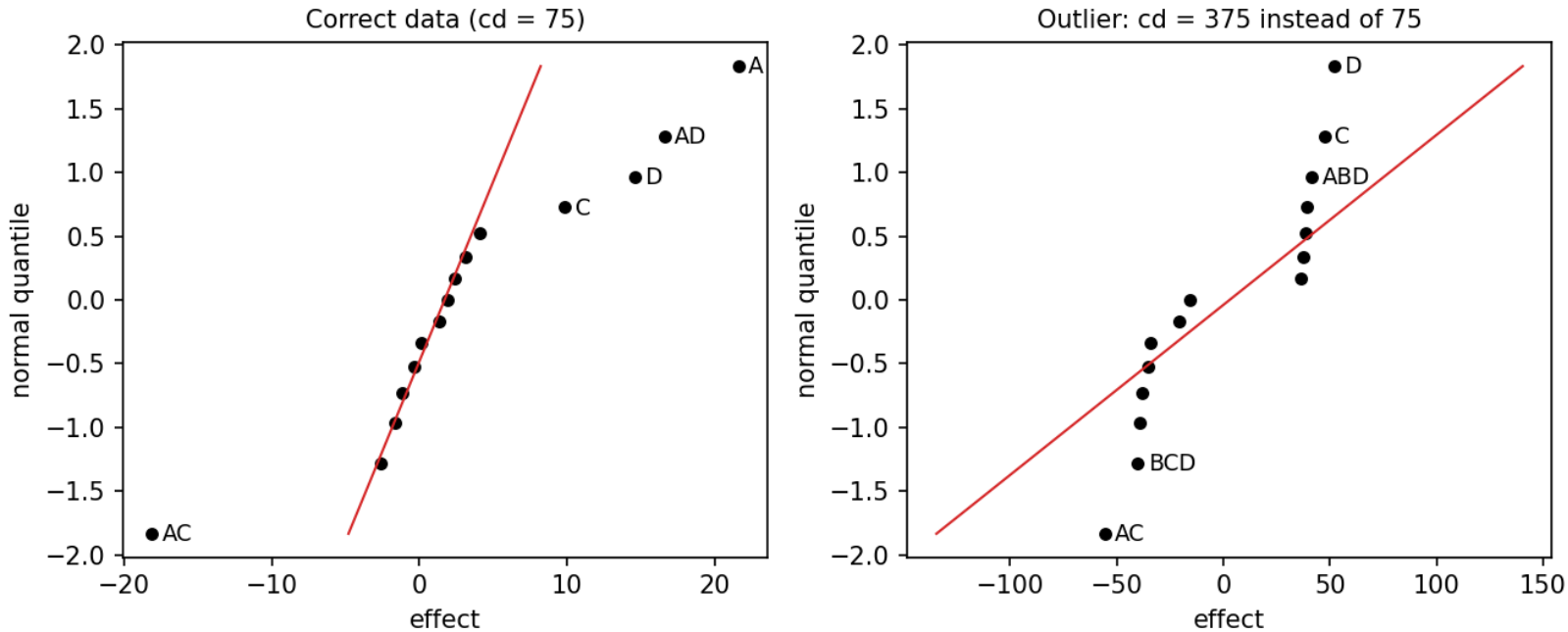
```
m3way = smf.ols("Rate ~ (A + B + C + D)**3", data=rate).fit()
```

	df	sum_sq	mean_sq	F	PR(>F)
A	1.0	1870.5625	1870.5625	247.3471	0.0404
C	1.0	390.0625	390.0625	51.5785	0.0881
D	1.0	855.5625	855.5625	113.1322	0.0597
A:C	1.0	1314.0625	1314.0625	173.7603	0.0482
A:D	1.0	1105.5625	1105.5625	146.1901	0.0525
...					
Residual	1.0	7.5625	7.5625	NaN	NaN

- An error term with 1 degree of freedom: $F_{0.05,1,1} = 161.4$, so even very large effects are hardly significant (here *C*, *D* and *AD* are not).
- The test has almost no power, and if *ABCD* were active, the error would be inflated. Prefer the normal plot / Lenth's method and then the projection.

The Effect of an Outlier on the Normal Plot of Effects

Suppose the run *cd* was recorded as 375 instead of 75 (Montgomery, Section 6.5). One wrong observation enters every effect with weight $\pm 300/8 = \pm 37.5$:



- The effects split into two groups around ± 37.5: a gap in the middle of the normal plot. The signs follow the column of *cd*.
- No straight line through the small effects; Lenth's $PSE = 58.3$, nothing is significant.
- Look for such patterns, check the data, and if the value cannot be corrected, analyse the data without it (or estimate it, Problem 6.34).

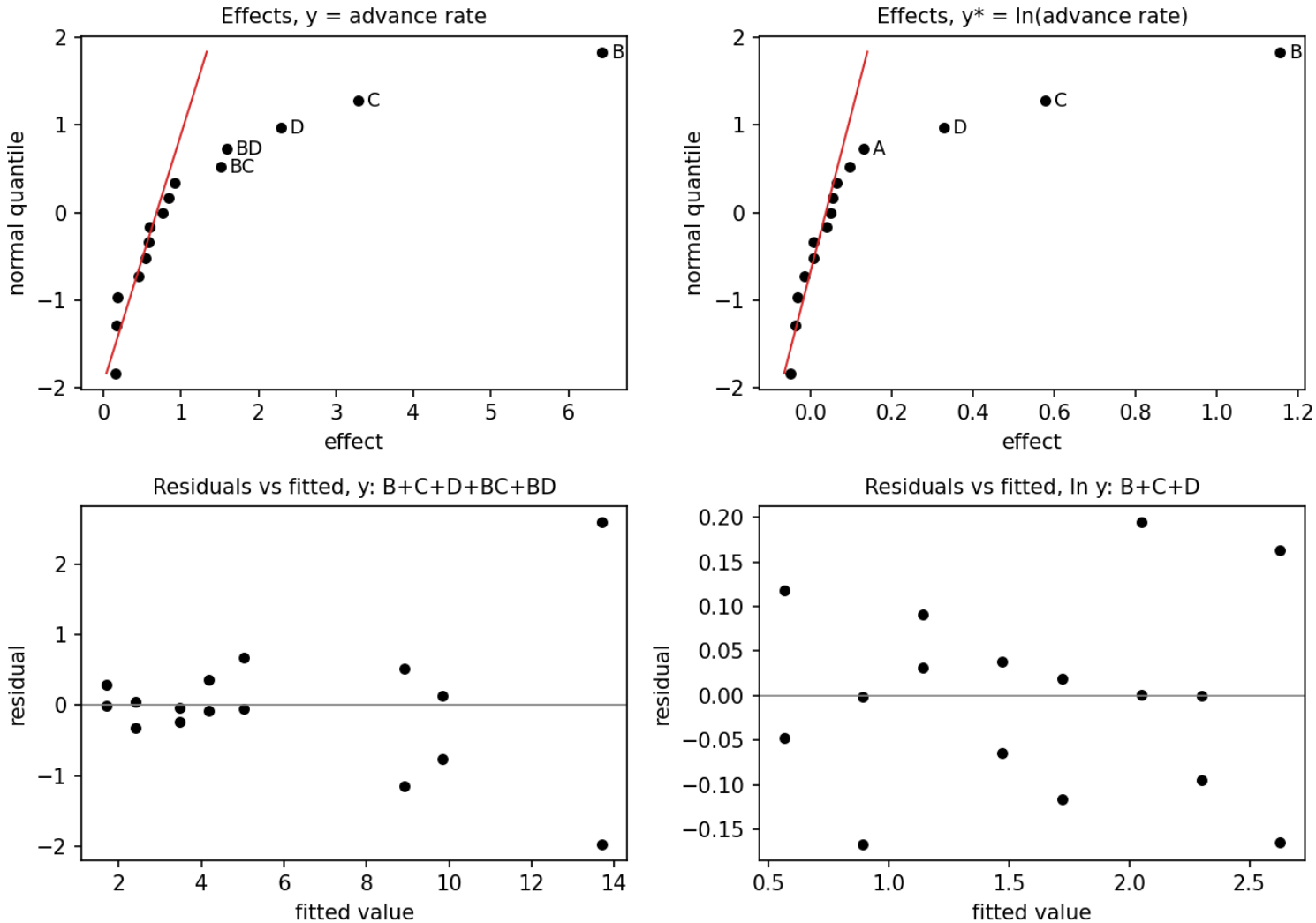
Data Transformation in a Factorial Design (Example 6.3)

Daniel (1976): a 2^4 design for the **advance rate of a drill**; factors drill load (A), flow rate (B), rotational speed (C) and type of drilling mud (D).

(1)	*a*	*b*	*ab*	*c*	*ac*	*bc*	*abc*
1.68	1.98	4.98	5.70	3.24	3.44	9.97	9.07
d	*ad*	*bd*	*abd*	*cd*	*acd*	*bcd*	*abcd*
2.07	2.44	7.77	9.43	4.09	4.53	11.75	16.30

- The normal plot of the effects suggests B, C, D and the interactions BC, BD.
- The residuals of this model show a clear **funnel**: the variance grows with the mean (Shapiro-Wilk $p = 0.04$).
- The response is a **rate**, effects are likely multiplicative: try $y^* = \ln y$.
- Box-Cox check (residual SS of the model $B + C + D$ on the scaled transformed response): $\lambda = 1$: 32.8, $\lambda = 0.5$: 10.6, $\lambda = 0$ (log): **4.2**, $\lambda = -0.5$: 7.9, $\lambda = -1$: 23.3.

Example 6.3 – Before and After the Log Transformation



After the log transformation only B, C, D are active, the interactions disappear, and the residuals show no funnel (Shapiro-Wilk $p = 0.81$).

Example 6.3 – Model for the Log Advance Rate

```
drill["log_rate"] = np.log(drill["rate"])
m_log = smf.ols("log_rate ~ B + C + D", data=drill).fit()
anova_lm(m_log).round(4)
```

	df	sum_sq	mean_sq	F	PR(>F)
B	1.0	5.3452	5.3452	371.4928	0.0000
C	1.0	1.3389	1.3389	93.0517	0.0000
D	1.0	0.4305	0.4305	29.9224	0.0001
Residual	12.0	0.1727	0.0144	NaN	NaN

- $R^2 = 7.115/7.287 = 0.976$. (Montgomery, Table 6.16, divides by the rounded $MS_E = 0.014$ and reports $F_B = 381.79$.)
- Expressing the data in the right metric simplified the structure: three main effects instead of three main effects plus two interactions. An interaction on the original scale can be just a consequence of a multiplicative model.
- Predictions are made on the log scale and transformed back by $\exp(\cdot)$ (this gives the median, not the mean, of the response).

Location and Dispersion Effects (Example 6.4)

A 2^4 design in a process producing interior panels for aircraft; response = average number of defects per panel in a press load. Factors: temperature (A), clamp time (B), resin flow (C), closing time (D).

- Normal plot of the effects: $A = 5.75$ and $C = -4.25$ are active (**location effects**), $R^2 = 0.77$. Lower temperature and higher resin flow reduce the defects.
- The residuals of the model $A + C$ plotted against B (clamp time) show a much **larger spread** at the high level of B: $S(B^+) = 2.72$, $S(B^-) = 0.82$.

Dispersion effect of factor i (Box and Meyer, 1986):

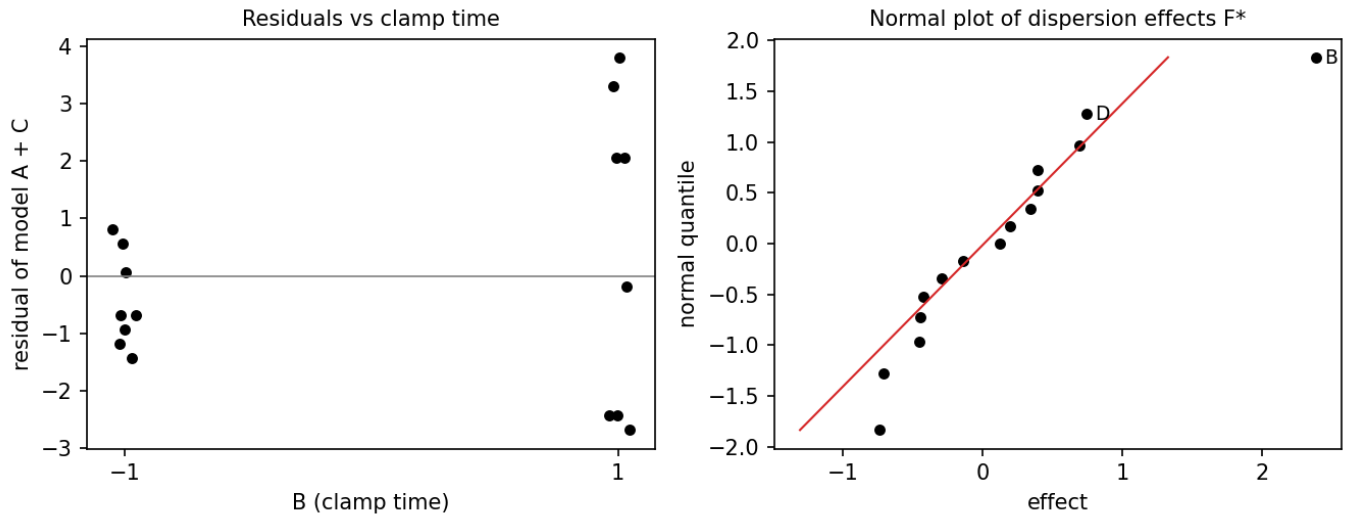
$$F_i^* = \ln \frac{S^2(i^+)}{S^2(i^-)}, \qquad i = 1, \ldots, 2^k - 1,$$

where $S(i^{\pm})$ is the standard deviation of the residuals at the runs where column i of the table of signs is $\pm$. If the two variances are equal, F_i^* is approximately normal, so we plot the F_i^* on a normal probability plot.

For clamp time $F_B^* = \ln(2.72^2/0.82^2) = 2.39$ (Montgomery: 2.37 with rounded standard deviations).

Example 6.4 – Dispersion Effects in Python

```
e = smf.ols("defects ~ A + C", data=panel).fit().resid
Fstar = {}
for k in range(1, 5):
    for t in itertools.combinations("ABCD", k):
        s = panel[list(t)].prod(axis=1)
        v_plus, v_minus = e[s == 1].var(), e[s == -1].var()
        Fstar["".join(t)] = np.log(v_plus / v_minus)
```



Clamp time (B) does not affect the mean but it does affect the variability: use the **low** clamp time. Recommendation: low temperature, high resin flow (location), low clamp time (dispersion).

Analysis of an Unreplicated 2^k Design – Summary

1. Plot the data (main effects, interactions) and compute all $2^k - 1$ effects.
2. Normal or half-normal plot of the effects, Lenth's *PSE*, *ME*, *SME* and a Pareto plot. Choose a tentative model (sparsity, hierarchy).
3. Fit the reduced model; the omitted effects form the error term. If some factors drop out completely, interpret it as a **projection** into a replicated design.
4. Check the residuals: normal plot, residuals vs fitted values, vs each factor (also the omitted ones – dispersion effects), vs run order.
5. If the residuals show a funnel, consider a **transformation** (log for rates and multiplicative effects, Box-Cox).
6. Watch for **outliers**: one wrong value distorts all effects and the normal plot.
7. Interpret the final model: effect plots, contour plots, recommended settings, and confirmation runs.

Today Exercise & Next Lecture

Ex06 – `code/01NAEX_Ex06.ipynb` (Python, Google Colab):

- The lecture examples in Python: filtration rate (effects, normal and half-normal plots, Lenth's method, projection, final model, contours), the outlier, drill advance rate (log transformation), panel process (dispersion effects).
- Assignment: Problems **6.17, 6.28 with 6.29, 6.31 with 6.32 and 6.36** (Montgomery, 8th ed.); **6.30** optional.

Assignment: start in class, finish at home as homework. Submit `HW/01NAEX_Ex06_HW_<Surname>.ipynb` by pull request. **Deadline: Sunday 8 November 2026.**

Team Project 1: deadline Sunday 8 November 2026.

Next lecture (9 November): blocking and confounding in the 2^k design, center points and the test for curvature (Montgomery, Sections 6.8, 7.1–7.6).